ComplexCodeEval: A Benchmark for Evaluating Large Code Models on More Complex Code

Jia Feng
University of Electronic Science and Technology of China
Shenzhen, China
jfeng@std.uestc.edu.cn

Jiachen Liu
Harbin Institute of Technology
Shenzhen, China
200110207@stu.hit.edu.cn

Cuiyun Gao*
Harbin Institute of Technology
Shenzhen, China
gaocuiyun@hit.edu.cn

Chun Yong Chong
Huawei, Hong Kong
Hong Kong, China
chunyong@ieee.org

Chaozheng Wang
The Chinese University of Hong Kong
Hong Kong, China
adf111178@gmail.com

Shan Gao
Huawei
Shenzhen, China
gaoshan17@huawei.com

Xin Xia
Huawei
Shenzhen, China
xin.xia@monash.edu

Abstract

In recent years, with the widespread attention of academia and industry on the application of large language models (LLMs) to code-related tasks, an increasing number of large code models (LCMs) have been proposed and corresponding evaluation benchmarks have continually emerged. Although existing evaluation benchmarks are helpful for comparing different LCMs, they may not reflect the performance of LCMs in various development scenarios. Specifically, they might evaluate model performance in only one type of scenario (e.g., code generation or code completion), whereas real development contexts are diverse and may involve multiple tasks such as code generation, code completion, API recommendation, and test function generation. Additionally, the questions may not originate from actual development practices, failing to capture the programming challenges faced by developers during the development process.

To address the aforementioned issues, we propose ComplexCodeEval, a new benchmark for evaluating the performance of LCMs in various development scenarios. ComplexCodeEval includes 3,897 Java samples from 1,055 high-star GitHub repositories and 7,184 Python samples from 2,107 high-star repositories. Each function sample in ComplexCodeEval contains multiple annotations (e.g., function signatures, docstrings and reference APIs) to accommodate various downstream tasks. Furthermore, to better reflect diverse development scenarios, each function sample is required to originate from a repository that depends on at least one selected library (based on popularity), and each function sample must invoke at least one API from the selected library. Additionally, each function sample has multiple timestamps to avoid data leakage. Based on ComplexCodeEval, we evaluate the performance of ten LCMs across four tasks (i.e., code generation, code completion, API recommendation, and test case generation) to explore their performance in complex development environments. Furthermore, we conduct an in-depth analysis of the impact of context and data leakage on model performance. Our experimental results reveal several key findings. For instance, LCMs exhibit varying performance across different coding tasks. Additionally, rich contextual information can greatly enhance the performance of LCMs. Moreover, using leaked data for evaluation may lead to an overestimation of model performance, resulting in inaccurate evaluation outcomes that deviate from the performance in practice.

CCS Concepts

• **Software and its engineering → Automatic programming**.

Keywords

Large Language Models, Code Intelligence, Benchmark

ACM Reference Format:
Jia Feng, Jiachen Liu, Cuiyun Gao, Chun Yong Chong, Chaozheng Wang, Shan Gao, and Xin Xia. 2024. ComplexCodeEval: A Benchmark for Evaluating Large Code Models on More Complex Code. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24), October 27-November 1, 2024, Sacramento, CA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3691620.3695552

1 Introduction

Code has become an important application area for LLMs [3, 4, 7, 13, 21, 22, 24, 25, 29, 30, 34, 37], leading to the emergence of numerous LCMs such as Codex [7], Copilot [2], and CodeLlama [30]. LCMs have been widely adopted for various tasks, including

*Corresponding author. The author is also affiliated with Peng Cheng Laboratory.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASE '24, October 27-November 1, 2024, Sacramento, CA, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1248-7/24/10
https://doi.org/10.1145/3691620.3695552

code generation, code completion, test case generation, and API recommendation. This widespread adoption of LCMs has remarkably advanced practical development by not only automating repetitive tasks, but also improving the quality of code and accelerating the overall software development process.

In order to understand the performance of LCMs on code, researchers have put a lot of effort into building evaluation benchmarks automatically or manually [4–7, 10, 14–16, 18–20, 32, 33, 35]. For instance, HumanEval [7] and MBPP [4] can effectively reflect the capabilities of LCMs in code generation. CrossCodeEval [9] assesses LCMs' performance in cross-file code completion, while CoderEval [35] and EvoCodeBench [20] evaluate the performance of LCMs in repository-level code generation. These benchmarks provide crucial references and guidance for the improvement and optimization of LCMs.

Although these evaluation benchmarks can effectively measure the performance of LCMs in some aspects, they may not accurately represent the programming challenges faced by developers in various development scenarios. Firstly, practical programming is diverse, whereas these benchmarks are evaluated on only one task (e.g., HumanEval focuses on code generation and CrossCodeEval focuses on code completion). Secondly, the samples used in these benchmarks may be manually crafted or sourced from a limited number of code repositories (e.g., EvoCodeBench from only 25 repositories), thereby covering only a narrow range of application domains and failing to effectively represent the challenges encountered in software development. Finally, these benchmarks may risk data leakage, as their samples could have been included in the training data (e.g., the samples of Concode [17] are sourced from GitHub repositories, and do not address data leakage concerns), potentially distorting the evaluation results.

Benchmark ComplexCodeEval. To address these limitations, we propose ComplexCodeEval in this paper. ComplexCodeEval is an evaluation benchmark designed to accommodate multiple downstream tasks, accurately reflect different programming environments, and deliberately avoid data leakage issues. ComplexCodeEval includes 3,897 Java samples from 1,055 code repositories and 7,184 Python samples from 2,107 code repositories. To ensure that ComplexCodeEval closely mirrors real-world development scenarios, we first screen 69 popular Java third-party frameworks and 55 popular Python third-party packages based on their SourceRank from Libraries.io [23]. These frameworks and packages cover a wide range of fields, such as web development, network communication, data processing and persistence, and security and encryption. Then, we select high-star repositories on GitHub that depend on these libraries and analyze them to track the usage of each library's API. Based on API usage frequency, we extract functions that rely on high-frequency APIs from these repositories as samples. To ensure ComplexCodeEval's suitability for multiple downstream tasks, we include various annotations for each sample, such as test cases, reference APIs, and docstrings. To avoid data leakage issues, we incorporate multiple timestamps for each function sample, including project creation time, file creation time, and function update time.

Empirical study. Based on ComplexCodeEval, we evaluate ten popular LCMs, including three families of open-source models (i.e., StarCoder2, CodeLlama, and Deepseek-Coder) with different sizes, as well as one closed-source model (i.e., GPT-3.5-Turbo). To assess the performance of LCMs, we evaluate them across four key tasks: code generation, code completion, API recommendation, and test case generation, respectively. We introduce various contextual information—such as file context and function dependencies—into the prompts to examine the impact of different context conditions on LCMs performance. To address the data leakage issue, we utilize file creation time as the basis for sample division, selecting samples from different timestamps to assess model performance. These experiments allow us to comprehensively understand the performance of LCMs across different tasks and context conditions, and evaluate the impact of data leakage on model performance.

Main findings and implications. Based on our experimental results, we have the following main findings: (1) *LCMs perform differently on different tasks.* In code generation of Java, CodeLlama-34B achieve the highest CodeBLEU score of 34.08, while in API recommendation of Java, CodeLlama-13B achieves the highest F1 score of 48.31. (2) *Rich context information can greatly enhance the performance of LCMs.* For instance, compared to the basic context, incorporating full contextual information increase the average CodeBLEU scores of all LCMs in Java and Python code generation by 70.73% and 31.90%, respectively. (3) *LCMs show inconsistent performance at different timestamps, specifically performing better on data that has been leaked.* For instance, compared to non-leaked data, in Java and Python code generation on leaked data, the average CodeBLEU scores of LCMs increase by 1.22 and 3.10, respectively.

ComplexCodeEval, its construction tools, and all experimental results have been open-sourced [1] to aid researchers and developers in better evaluating and optimizing LCMs. To address potential data leakage with the emergence of new models, we plan to regularly update our benchmark every six months, similar to the approach taken by LiveCodeBench [18], to ensure compatibility with the latest mainstream LCMs.

In summary, this paper makes the following contributions:

- We introduce ComplexCodeEval, a benchmark suitable for multiple downstream tasks, which can reflect the various programming environment and deliberately avoid data leakage problems.
- We evaluate the performance of ten popular models on four tasks, revealing the programming capabilities of LCMs more comprehensively.
- We reveal the impact of context and time on the performance of LCMs in code generation and code completion.

2 Background And Related Work

2.1 Large Code Models

LLMs are widely applied in the coding domain for diverse tasks such as code generation, code completion, test case generation, and API recommendation. A specialized subset of LLMs, known as LCMs, are trained on code corpora and instructions. Notable examples of LCMs, such as DeepSeek-Coder [13] and Starcoder2 [21], supporting context windows of up to 16k tokens. These models claim proficiency in project-level code generation and completion tasks, demonstrating strong performance across several established benchmarks. Additionally, numerous LCMs have been proposed, including Codellama [30], WizardCoder [25], and others [3, 34, 37].

Table 1: The comparison between existing benchmarks and ComplexCodeEval. CC, M/A, NL, and ADL indicate cyclomatic complexity, Manual/Automated, nature language and avoiding data leak, respectively.

Benchmark	Code Distribution					Annotation	M/A	ADL	Task
	#Repos	#Samples	#LOC	#Tokens	#CC				
Concode	-	2,000	1.00	30.59	1.49	NL, Code	A	✗	CG
CoNaLA	-	500	1.11	14.28	0.00	NL, Code	A	✔	CG
methods2test	1,017	78,388	-	-	-	Code, Test	A	✗	TCG
APIBench-C	3,700	-	-	-	-	Code	A	✗	AR
APPS	-	10,000	16.46	140.61	2.27	NL, Code, Example	A	✔	CG
MBPP	-	974	6.68	48.60	2.79	NL, Code	M	✔	CG
HumanEval	-	164	8.95	57.57	3.59	NL, Code, Signature	M	✔	CG
DS-1000	-	1,000	3.66	43.34	0.52	NL, Code, Context	A	✔	CG
CrossCodeEval	1,002	9,928	-	-	-	NL, Code, Context	A	✔	CC
ClassEval	-	100	45.70	123.70	-	NL, Code, Depend	M	✔	CG
CoderEval-Java	10	230	9.17	66.42	3.10	NL, Code, Depend, Context	M	✗	CG
CoderEval-Python	43	230	20.64	119.26	4.52				
ComplexCodeEval-Java	1,055	3,897	31.72	251.45	6.52	NL, Code, Depend, Context, Repository, Path, Test, Time	A	✔	CG, CC, TCG, AR
ComplexCodeEval-Python	2,107	7,184	38.21	293.64	7.57				

2.2 Benchmarks for Code-Related Tasks

Existing benchmarks for code-related tasks are often oriented towards different downstream tasks. In code generation tasks, LCMs generate code snippets or entire functions based on given nature language descriptions, streamlining the initial development process. Benchmarks for code generation, such as HumanEval [7] and MBPP [4], evaluate the performance of LCMs in generating Python functions from natural language descriptions and function signatures. However, these benchmarks are limited to standalone functions (i.e., the function has no external dependencies). DS-1000 [19] expands the scope to tasks involving third-party data science libraries but is restricted to seven libraries with an average code length of 3.66 lines. CoderEval [35] addresses repository-level code generation for both standalone and non-standalone functions, but its evaluation datasets come from only a small number of code repositories (CoderEval is curated from 53 code repositories in total).

Apart from code generation, code completion, which aims to predict the next part of a code snippet, is also another popular downstream task that has gained a lot of attention. A recently proposed benchmark for this task is CrossCodeEval [9], which is used to evaluate LCMs' capabilities in cross-file code completion. However, in real-world software development, the application typically utilize built-in or third-party libraries to realize certain functions. Hence, when developers are utilizing LCMs to perform code completion, there is a high chance that APIs are involved in the code completion task, which is commonly known as API recommendation. API recommendation aids developers in finding and using the most appropriate APIs for their specific needs. Benchmarks for API recommendation tasks, such as APIbench [27], facilitate the assessment of query-based and code-based API recommendation scenarios. Apart from that, another important downstream task is test case generation, which leverages LCMs to automatically create test cases, ensuring that the code functions as expected and helping to maintain software reliability. Method2Test [31] is a dataset for test case generation, containing 780,944 instances from 9,410 Java projects.

Table 1 presents twelve different benchmarks, detailing their size, code scale, code complexity, annotation information, collection methods, and target tasks. For comparison, our constructed benchmark, ComplexCodeEval, is displayed in the last two rows. While the aforementioned benchmarks have gained significant popularity among researchers for evaluating the performance of LCMs on specific downstream tasks, they exhibit three major limitations. First, they typically focus on only a few specific code repositories or are based on synthetic datasets created manually, rather than real-world repositories, as seen in HumanEval, MBPP, and CoderEval. This lack of broader repository coverage may result in discrepancies between model performance in practical applications and benchmark results. Second, current benchmarks primarily assess isolated code-related tasks, failing to provide a holistic evaluation of LCMs' capabilities across multiple dimensions. Lastly, several benchmarks, including CoderEval, Method2Test, and APIbench, have not adequately addressed data leakage issues, potentially leading to evaluation biases.

To address these limitations, we introduce ComplexCodeEval, a novel benchmark that provides comprehensive evaluations across various downstream coding tasks, including code generation, API recommendation, and test case generation, making it a more robust and versatile tool for assessing the diverse capabilities of LCMs. ComplexCodeEval is derived from a wide range of GitHub repositories to ensure it reflects real-world coding scenarios, and it incorporates timestamps to mitigate data leakage. Compared to existing benchmarks, ComplexCodeEval offers broader coverage and more comprehensive evaluations, providing a better reflection of model performance in complex development environments.

3 ComplexCodeEval

This section is divided into three subsections. The first two subsections outline the construction process of ComplexCodeEval (as illustrated in Figure 1), covering data collection and dataset generation. The third subsection provides a detailed overview of the key features of ComplexCodeEval.

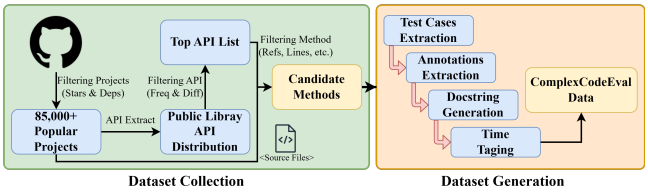

Figure 1: The process of ComplexCodeEval construction.

3.1 Dataset Collection

3.1.1 Library Selection. To ensure that ComplexCodeEval is close to complicated development environments and reflects diversity, we collect popular libraries from Maven (PyPI) through Libraries.io. First, we access the Libraries.io API and sort the libraries based on their SourceRank. SourceRank takes into account multiple factors, including repository activity (e.g., commit frequency, version releases, and issue and pull request handling) and package popularity (e.g., download counts, number of dependencies, stars, and forks). For Python and Java, we select the top 100 most popular libraries respectively. Next, we manually check the collected libraries, excluding tools that could not be integrated through library dependencies (e.g., command-line tools), ultimately filtering down to 69 popular Java frameworks and 55 popular Python packages. These libraries span several broad domains, for instance, web development and network communication (e.g., Java's Spring and Python's Django), data processing and persistence (e.g., Java's MyBatis and Python's pandas), and distributed systems and microservice architectures (e.g., Java's Apache Dubbo and Python's celery).

3.1.2 Repository Selection. We collect Java and Python repositories from GitHub with over 99 stars and obtain more than 85,000 repositories. Then we analyze the dependencies of each repository. Specifically, we first extract the external libraries that the code repositories depend on by parsing all the configuration files in the repository (e.g., Java's pom.xml; Python's requirements.txt and setup.py). To ensure that the extracted dependencies are more comprehensive and accurate, we also retrieve dependencies from the repositories' corresponding Software Bill of Materials (SBOM) on GitHub, combining both sources to obtain the full dependency information. Based on the dependency information, we retain the repositories that depend on the selected libraries. Ultimately, we retain 9,169 Java projects and 27,178 Python projects.

3.1.3 Candidate Methods Extraction. We first manually define API filtering rules for each selected library, focusing on the common prefixes of the APIs within each library. These rules are implemented by specifying Accept and Reject fields for each library. The Accept field lists common prefixes for APIs belonging to the target library, while the Reject field excludes APIs that share similar prefixes but belong to different libraries. Next, for each selected project, we construct an abstract syntax tree (AST) for every code file. By traversing the ASTs and applying the filtering rules, we determine which APIs from the selected libraries are called in each code file and the corresponding frequencies. We then aggregate this information to obtain the API usage frequencies across all projects, selecting the top 100 (if available) most frequently called APIs for each library. For the projects that call these APIs, we construct ASTs for their code files and model each project, retaining all test functions, as well as non-test functions that have more than 10 lines of code.

```
1. ...
2. from django_extensions.admin.filter import NullFieldListFilter, ...
3. ...
4. class NullFieldListFilterTests(BaseFieldFilter):
5.     ...
6.     def test_choices(self):
7.         ...
8.         filter_spec = NullFieldListFilter(...)
9.         result = filter_spec.choices(m_cl)
10.         ...
11. ...
```

Figure 2: An example of a Python test function. The *choices* function is invoked by *filter_spec* (line 9), where *filter_spec* is an instance of *NullFieldListFilter*. Considering that *NullFieldListFilter* is imported from the *django_extensions.admin.filter.NullFieldListFilter* class (line 2), the original path of *choices* is set as the imported class.

3.2 Dataset Generation

3.2.1 Test Cases Extraction. We utilize the project modeling obtained from the *Candidate Methods Extraction* stage to match functions with their corresponding test functions. Specifically, for Java projects, we match them through the following steps: (1) Traverse each non-test file in the project and match it with its corresponding test file by filename. (2) Within each matched non-test file, traverse each function entity and match it with the corresponding test function by function name. (3) In each matched test function, traverse all its function calls and determine whether it calls the corresponding non-test function by function name and parameters. Through these steps, we ultimately obtain the pairs of functions and test functions.

Since Python test functions do not have a uniform standard, and their parameter types and quantities are indeterminate, the same matching method used for Java projects cannot be directly applied. Therefore, we take the following steps for Python projects: (1) Match non-test files and test files by filename. (2) In the matched test files, traverse each test function. (3) For each test function, reconstruct the original path of each function call (e.g., as shown in Figure 2, the original path of the *choices* function is *django_extensions.admin.filter.NullFieldListFilter.choices*). (4) Traverse each function in the corresponding non-test file, match the function path with the original path. Through these steps, we obtain the pairs of functions and test functions.

3.2.2 Annotations Extraction. To ensure that ComplexCodeEval is suitable for multiple downstream tasks, we make considerable efforts to extract metadata related to different downstream tasks. Specifically, we use the functions extracted from the *Test Cases Extraction* stage as function samples and further analyze the corresponding repositories for each function, annotating each sample with multiple attributes, such as: reference API (i.e., the popular APIs called by the samples), import information, context, docstring (i.e., natural language description of the function), function signature, code implementation, test function. An example of a sample can be seen in Figure 3.

To ensure the diversity of ComplexCodeEval samples, we ensure that each sample under an API comes from different projects (i.e., the same API can only extract one sample in a project). When the similarity between samples under one API and another API

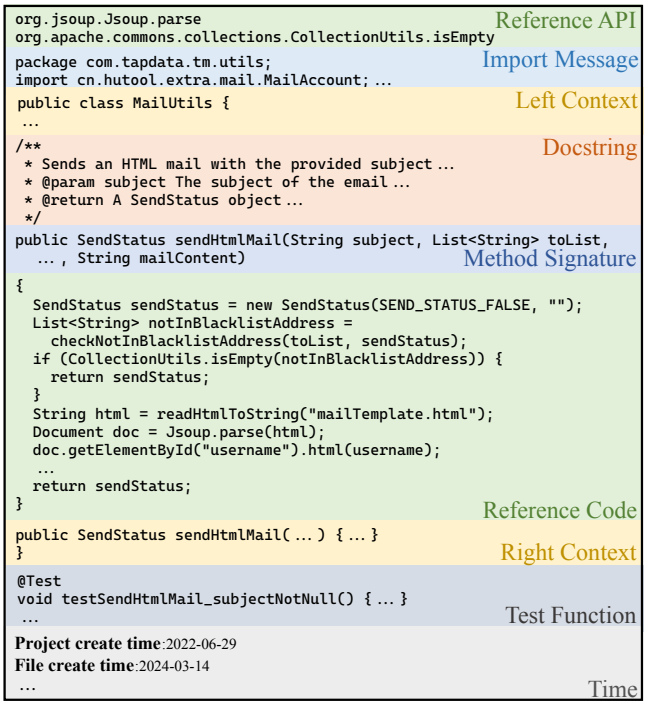

Figure 3: An example of ComplexCodeEval.

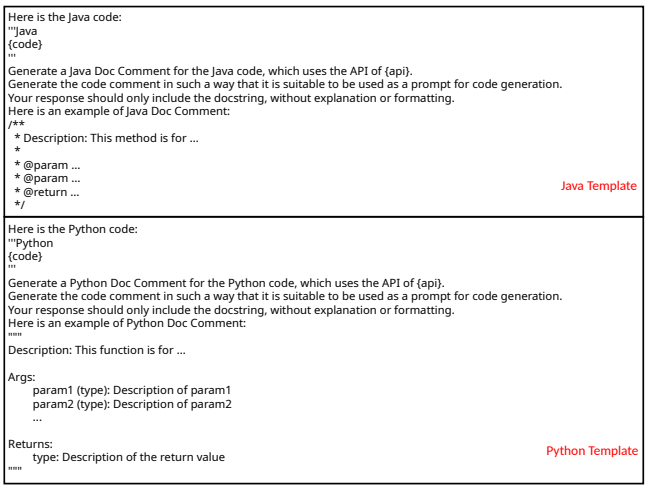

Figure 4: The prompt template used for generating docstrings.

exceeds half of its own sample count, we randomly remove one of the APIs and its corresponding samples. Furthermore, we reselect high-frequency APIs within the corresponding frameworks based on API call frequency for sample extraction. Finally, we perform deduplication of function implementations for the samples using the exact matching method.

3.2.3 Docstring Generation. Due to the impact of prompt quality on the code generated by LCMs, it is essential to ensure that the docstrings provided in ComplexCodeEval comprehensively and accurately describe the functions (where docstring is usually used as the prompt in benchmarks). Therefore, we investigate the quality

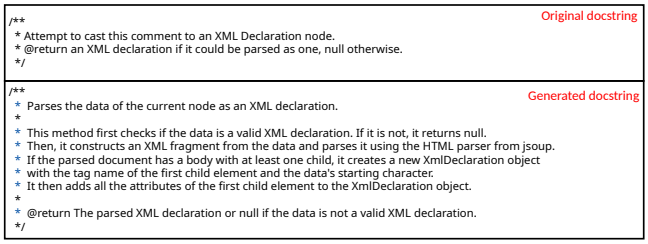

Figure 5: An example of the original docstring and the LLM-generated docstring.

of the original docstrings. Specifically, we sample and check the original docstrings (an example can be seen in Figure 5) and find that their quality varied widely. Many docstrings are relatively short, containing only a brief description of the target function, and are often ambiguous and incomplete.

Manually writing docstring is both time-consuming and labor-intensive. Inspired by the strong capabilities of LLMs in generating code comments [12], we use a LCM (DeepSeek-Coder-33B) to generate docstrinsg for all of the candidate questions in ComplexCodeEval (see Section 6.1 for further analysis). Specifically, we design an initial prompt template to instruct the model to generate docstrings, and we manually evaluate the effectiveness of the prompt template through an iterative process. To ensure its robustness, we randomly sample 100 examples, assessing the generated docstrings based on three criteria: (1) The docstrings should only include necessary descriptions without extraneous content, (2) the parameters and return values must accurately match those of the original function, and (3) the docstring should clearly and correctly reflect the function's purpose and content, as verified independently by the first and second authors. We repeat this process, refining the prompt template until it consistently produces high-quality docstrings. The final prompt template can be seen in Figure 4. Additionally, to maintain the authenticity of the samples, we retain the original docstring and add the LLM-generated docstring (an example of docstring can be seen in Figure 5) for each sample.

3.2.4 Time tagging. Data leakage is one of the most crucial problems to address when it comes to the LLM/LCM benchmarks. It is imperative to ensure that the questions in the benchmark are not seen by the model. We adopt a flexible approach by adding timestamps to each function sample to dynamically address the problem of data leakage. Initially, we consider using the repository creation time as the time information for each function sample. However, upon analyzing the distribution of function samples based on repository creation and update times, we find that using only repository time information did not adequately reflect the function sample distribution. Therefore, we obtain more detailed time information by analyzing the git commit history of each sample's corresponding GitHub repository. Specifically, we use the earliest commit time of the file corresponding to the sample as the file creation time, the most recent commit time of the file as the file update time and the most recent commit time involving additions or modifications to the function's code as the function update time (deletions of function code were not considered updates). By using the timestamp information, we can purposefully avoid data leakage issues by considering the model's knowledge cut-off date.

Table 2: Selected LCMs.

Model	size	time	instruct	base
DeepSeek Coder	33B	2023-01-01	✓	✓
DeepSeek Coder	6.7B	2023-01-01	✓	✓
DeepSeek Coder	1.3B	2023-01-01	✓	✓
StarCoder 2	15B	2023-09-14	✓	✓
StarCoder 2	7B	2023-09-14	✗	✓
StarCoder 2	3B	2023-09-14	✗	✓
CodeLLaMa	34B	2023-01-01	✓	✓
CodeLLaMa	13B	2023-01-01	✓	✓
CodeLLaMa	7B	2023-01-01	✓	✓
GPT-3.5-Turbo	N/A	2021-10-01	✓	✗

3.3 Benchmark Characteristic

Through the aforementioned steps, we construct ComplexCodeEval. Its detailed characteristics are as follows:

Flexibility. ComplexCodeEval contains rich information, including context, reference APIs, and test functions (as shown in Figure 3). These elements can be flexibly combined, enabling ComplexCodeEval to be used for various code tasks such as code generation, code completion, test case generation, and API recommendation. Additionally, various supplementary information can be incorporated into each task to explore the performance of LCMs with different data inputs. Moreover, ComplexCodeEval provides multiple timestamps, including the creation and update times at the project, file, and method levels. We can leverage this information to adjust the data to meet our needs, such as avoiding data leakage issues.

Scale. Table 1 presents the overall data scale information of ComplexCodeEval for comparison with other benchmarks. The results show that ComplexCodeEval comprises 3,897 samples from 1,055 Java projects and 7,184 samples from 2,107 Python projects. The average lines of code in ComplexCodeEval-Python reaches 38.21, second only to ClassEval's 45.70 (notice that ClassEval is at the class level, containing multiple functions, whereas ComplexCodeEval is at the function level, containing only a single function). Furthermore, the average cyclomatic complexity of the Python and Java samples in ComplexCodeEval are 7.57 and 6.52, respectively, higher than all existing datasets. These results indicate the challenge of tasks involved in our benchmark.

4 Experimental setup

With ComplexCodeEval, our experiment aims to answer the following research questions:

- **RQ1**: How do LCMs perform on ComplexCodeEval varies across different tasks?
- **RQ2**: How does different contextual information impact LCMs performance on these tasks?
- **RQ3**: How does data leakage affect the effectiveness of LCMs on these tasks?

4.1 Model Selection

As shown in Table 2, we select ten distinct LCMs to assess their performance on code intelligence tasks. These models include four categories: the StarCoder2 family (SC2), the Codellama family (CL), the DeepSeek-Coder family (DSC), and one closed-source model (GPT-3.5-Turbo). These models encompass various scales and training objectives, ranging from small-scale to large-scale models, covering state-of-the-art large code model technologies. The selection of models is based on their performance in existing research and their availability.

4.2 Task Design

To comprehensively assess the performance of LCMs, we identify four representative tasks: code generation, code completion, test case generation, and API recommendation. Below, we describe each of these tasks in detail.

Code Generation is a common task to evaluate the capabilities of LCMs to translate natural language into code. In our experiment, the model is furnished with a function signature and its corresponding docstring, and is required to generate a correct implementation. This task simulates a scenario where developers write documentation describing the functionality and signature of a function, using LCMs to automatically generate the function's implementation code to enhance development efficiency.

Code Completion is employed to assess the code completion capabilities of LCMs, which showcases how LCMs can be used to predict and complete code based on an initial portion provided by developers, improving programming efficiency and code quality. Inspired by previous research [26], we provide the initial half of the function to the LCMs, and the LCMs need to complete the remaining portion.

Test Case Generation aims to evaluate the LCMs' comprehension of the original function's functionality. In this task, the model is provided with the original function and the function signature of the test cases. The LCM is subsequently required to generate the specified test case to assess specific functionality or segments of the code. This scenario illustrates how LCMs can automatically create test cases that validate code functionalities, reducing manual effort and enhancing test coverage.

API Recommendation tasks are generally categorized into query-based API recommendation and code-based API recommendation. This paper focuses on the latter, where the model is provided with a code snippet and is required to recommend an appropriate public library API. This scenario highlights how LCMs can help developers quickly find suitable APIs based on the provided code snippet, enhancing development efficiency and reducing the time spent searching and selecting APIs.

4.3 Annotation selection

To accommodate the requirements of the four selected tasks, we select the following annotations from ComplexCodeEval, accompanied by their corresponding explanations.

- **Reference API (A)** includes the public library APIs invoked by the target function. It serves as a reference during the evaluation phase of API recommendation tasks.
- **Import Information (I)** encompasses the libraries imported within the file and the package information of the file itself. It can be provided as supplementary information to LCMs for inference across various tasks.
- **File Context (C)** consists of all code snippets within the file that are outside the target function. It can be used as additional information to aid LCMs in reasoning.

- **Docstring (D)** refers to the documentation comments of the target function, typically describing the function's logic, parameters, and return values. It is used as part of the prompt in code generation tasks.
- **Method Signature (S)** is the function signature of the target function, also included as part of the prompt in code generation tasks.
- **Reference Code (R)** is the reference implementation of the target function from the original project. It is used as a ground truth in the evaluation phase of code generation tasks and can also serve as a prompt for code completion, test case generation, and API recommendation tasks.
- **Code Snippet (CS)** refers to the lines of code preceding the target completion line, which are used for code completion.
- **Test Function (T)** is the corresponding test function for the target function from the original project. It is used as a reference during the evaluation phase of test case generation tasks.
- **Dependency (Dep)** includes all external dependency functions within the target function. It can be provided as additional information for LCMs to reason across various tasks.
- **Timestamp (TS)** contains timestamps such as project creation and file creation dates, which can be utilized for dataset partitioning.

4.4 Evaluation Metrics

For the first three tasks (code generation, code completion, test case generation), we use CodeBLEU [28] (or BLEU [11] in code completion) and ES (Edit Similarity) as the evaluation metric. CodeBLEU is specifically designed to evaluate the quality of code generation, considering multiple aspects like BLEU, AST (abstract syntax tree), data flow, and naming entities, providing a comprehensive reflection of generated code quality. The computation of CodeBLEU score includes the following steps:

- **BLEU**: Evaluates n-gram and weighted n-gram overlap between generated code and reference code.
- **Syntax Match**: Compares the abstract syntax tree structures of the generated and reference codes.
- **Data Flow Match**: Analyzes data flow similarity between generated and reference codes.
- **Average Score**: Calculates the average value of the aforementioned scores.

For the API recommendation task, we use Recall and F1 to evaluate the quality of the APIs recommended by the model.

4.5 Implementation Details

Our experiments are run on a server with two A100-80G GPUs. To eliminate the effects of random sampling, we adopted a greedy decoding strategy. Specifically, we set the temperature to 0, top_p to 1, and max_tokens to 1000 (with the API-recommended setting being 10). For each task, we randomly select 84 to 100 pieces of samples for both Java and Python from the dataset, specifically from data after September 15, 2023, based on the model's knowledge cutoff time (as shown in Table 2).

5 Evaluation Result

5.1 RQ1: Performance of LCMs on ComplexCodeEval

In Table 3, we present the performance of ten LCMs on four different tasks in Python and Java programming languages. From Table 3, we can derive the following observations: **LCMs still exhibit certain limitations in intricate development scenarios.** Experimental results indicate that LCMs generally exhibit suboptimal performance across all four tasks. For instance, in the code generation task, the best-performing model, Codellama-34B, achieves CodeBLEU scores of only 27.54 for Python and 34.08 for Java. These scores are benchmarked against the original developer-written code in the collected projects. The low CodeBLEU scores highlight a significant disparity between the code generated by LCMs and the original code. Similarly, in the code completion task, the highest BLEU scores achieved are 19.62 for Python and 31.86 for Java. In the test case generation task, the top CodeBLEU scores are 22.87 and 29.90, respectively. For the API recommendation task, the highest F1 scores are 52.24 for Python and 48.31 for Java. These results indicate that LCMs are still far from being viable for real-world development tasks and suggest the need for supplementary techniques to enhance reasoning capabilities and further advancements in model intelligence.

Each model exhibits unique capabilities depending on the specific tasks and programming languages. We find that each model excels in different tasks during the evaluation process. For instance, although Codellama-34B performs the best in the code generation task for both Python and Java, the best-performing model in code completion is DeepSeek-Coder-33B. In test case generation, Codellama-34B achieves the highest score in Java, while DeepSeek-Coder-6.7B outperforms all other models in Python with a CodeBLEU score of 22.87. For the API recommendation task, Codellama-34B achieves the highest score of 52.24 on the Python dataset but performs poorly on the Java dataset with an F1 score of only 38.92, far inferior to Codellama-13B. Although larger models generally perform better on tasks, these differences indicate that it is difficult to generalize which model is the best across different tasks and languages. A comprehensive evaluation of the models' capabilities in various aspects is necessary, and in practical applications, the most suitable model should be selected based on specific needs.

Finding 1: Current models continue to display limitations in intricate development scenarios. Furthermore, the performance of different models varies across programming languages and tasks, highlighting the need for a thorough evaluation of each model's capabilities across multiple dimensions.

5.2 RQ2: Impact of Contextual Information on LCM's Performance

In this section, we explore the impact of context on the performance of LCMs in both code generation and code completion to reflect the necessity of context in complicated development scenarios.

Code Generation. As shown in Table 4, under the most basic context conditions (including only docstring and signature, referred to as D+S in the table), the average CodeBLEU scores for Python

Table 3: Performance of various LCMs on ComplexCodeEval. Bold numbers on a gray background indicate the maximum values.

Metrics	Language	DSC-33B	DSC-6.7B	DSC-1.3B	SC2-15B	SC2-7B	SC2-3B	CL-34B	CL-13B	CL-7B	GPT-3.5	σ
Code Generation												
CodeBLEU	Java	33.96	32.23	26.91	32.59	30.83	22.46	**34.08**	28.98	29.58	19.92	4.79
ES	Java	35.64	34.24	29.70	35.90	31.55	26.38	**36.61**	34.04	33.61	21.66	4.78
CodeBLEU	Python	26.29	25.72	21.90	26.70	27.19	24.68	**27.54**	19.95	25.90	26.09	2.43
ES	Python	30.45	29.70	26.00	32.15	32.13	30.17	**32.93**	26.92	31.09	31.05	2.24
Code Completion												
BLEU	Java	**31.86**	30.81	20.53	21.67	16.82	15.91	22.36	21.28	26.09	20.31	5.33
ES	Java	33.09	**34.17**	26.38	24.22	17.35	20.54	26.78	25.83	29.34	24.21	5.16
BLEU	Python	16.70	15.33	10.69	12.18	11.32	11.50	13.79	10.97	11.97	**19.62**	2.95
ES	Python	**29.18**	28.45	21.77	22.89	19.47	19.91	25.48	22.79	22.99	19.61	3.47
Test Case Generation												
CodeBLEU	Java	29.33	27.05	19.27	25.27	2.57	13.10	**29.90**	25.67	23.36	25.58	8.46
ES	Java	29.13	29.10	22.99	26.04	5.24	15.96	29.39	**29.64**	26.24	28.54	7.89
CodeBLEU	Python	19.40	**22.87**	9.79	18.27	14.32	13.53	21.40	20.80	19.41	17.67	4.05
ES	Python	24.95	**28.65**	13.38	25.92	19.74	18.37	28.00	27.70	24.80	25.54	4.95
API Recommendation												
F1	Java	44.57	41.85	34.00	32.20	42.15	39.43	38.92	**48.31**	45.75	47.21	5.38
Recall	Java	44.72	42.00	33.71	32.29	42.18	39.57	38.73	**47.56**	45.75	46.67	5.26
F1	Python	49.76	43.18	37.47	47.48	45.61	41.98	**52.24**	42.20	36.09	43.88	5.04
Recall	Python	49.84	42.97	37.35	48.03	46.31	42.29	**52.14**	42.83	36.36	44.16	5.05

Table 4: Performance of LCMs in code generation across different contexts. Bold numbers on a gray background indicate the maximum values and red indicates the growth relative to the baseline.

Annotations	Language	DSC-33B	DSC-6.7B	DSC-1.3B	SC2-15B	SC2-7B	SC2-3B	CL-34B	CL-13B	CL-7B	Average
CodeBLEU											
D+S	Python	26.29	25.72	21.90	26.70	27.19	24.68	**27.54**	19.95	25.90	25.10
D+S+I	Python	**31.13**	29.32	22.83	29.76	28.19	27.83	31.04	25.04	28.91	28.23 (↑12.48%)
D+S+C	Python	34.80	32.01	26.02	**35.34**	28.77	30.63	31.52	26.49	29.72	30.59 (↑21.88%)
D+S+C+Dep	Python	**38.52**	35.68	28.52	35.87	32.18	33.17	35.15	26.40	32.44	33.10 (↑31.90%)
D+S	Java	33.96	32.23	26.91	32.59	22.01	22.46	**34.08**	28.98	29.58	29.20
D+S+I	Java	**42.52**	38.89	31.42	38.36	24.37	26.02	38.80	36.87	35.04	34.70 (↑18.83%)
D+S+C	Java	46.90	**47.21**	36.94	46.39	30.83	35.72	46.07	43.90	36.45	41.16 (↑40.95%)
D+S+C+Dep	Java	**56.69**	56.33	42.96	53.76	43.64	46.68	55.35	43.75	49.53	49.85 (↑70.73%)
Edit Similarity											
D+S	Python	30.45	29.70	26.00	32.15	32.13	30.17	**32.93**	26.92	31.09	30.17
D+S+I	Python	35.57	32.30	26.20	33.99	30.97	32.11	**36.31**	31.16	34.83	32.60 (↑8.04%)
D+S+C	Python	**39.64**	36.54	29.95	39.39	33.08	33.95	35.94	31.98	34.10	34.95 (↑15.83%)
D+S+C+Dep	Python	**42.06**	39.42	32.82	40.29	35.85	36.93	39.08	32.39	35.84	37.19 (↑23.25%)
D+S	Java	35.64	34.24	29.70	35.90	24.86	26.38	**36.61**	34.04	33.61	32.33
D+S+I	Java	**41.73**	37.96	31.37	36.42	24.72	27.32	39.17	39.04	34.53	34.70 (↑7.34%)
D+S+C	Java	**45.41**	43.95	36.04	44.29	31.55	35.88	44.10	44.36	36.31	40.21 (↑24.39%)
D+S+C+Dep	Java	**55.16**	51.78	41.83	53.60	44.05	45.30	52.76	43.81	48.41	48.52 (↑50.08%)

and Java are only 25.10 and 29.20, with average ES scores of 30.17 and 32.33, respectively. This indicates that the model struggles to achieve the desired effect in actual development with just basic information. After adding import information, the average CodeBLEU scores for Python and Java increase to 28.23 and 34.70, representing a 12.48% and 18.83% improvement, respectively, compared to using only basic information. The ES scores also increase by 8.04% and 7.34%, respectively. Furthermore, when file context containing import statements is added, the CodeBLEU scores increase by 21.88% and 40.95%, and the corresponding ES scores improve by 15.83% and 24.39%. Finally, after including function dependencies, the model's performance sees the greatest improvement, with CodeBLEU scores increasing by 31.90% and 70.73%. Additionally, we find that with more comprehensive contextual information, the performance of

Table 5: Performance of LCMs in code completion across different contexts. Bold numbers on a gray background indicate the maximum values and red indicates the growth relative to the baseline.

Annotations	Language	DSC-33B	DSC-6.7B	DSC-1.3B	SC2-15B	SC2-7B	SC2-3B	CL-34B	CL-13B	CL-7B	Average
BLEU											
CS	Python	**16.70**	15.33	10.69	12.18	11.32	11.50	13.79	10.97	11.97	12.72
CS+D	Python	**23.08**	21.21	14.92	17.68	14.11	13.35	15.95	17.04	15.16	16.94 (↑33.25%)
CS+D+C	Python	28.87	**30.63**	16.38	24.63	23.37	15.92	25.03	20.79	19.85	23.41 (↑84.12%)
CS	Java	**31.86**	30.81	20.53	21.67	16.82	15.91	22.36	21.28	26.09	23.04
CS+D	Java	**40.58**	39.14	31.71	31.21	15.45	19.16	33.31	29.00	32.83	30.27 (↑31.38%)
CS+D+C	Java	**51.96**	51.11	40.56	38.89	30.09	29.90	49.51	29.37	45.25	40.88 (↑77.44%)
Edit Similarity											
CS	Python	**29.18**	28.45	21.77	22.89	19.47	19.91	25.48	22.79	22.99	23.66
CS+D	Python	**34.49**	32.71	24.76	28.63	20.90	20.55	26.44	26.50	27.14	26.90 (↑13.71%)
CS+D+C	Python	37.55	**38.59**	25.04	33.09	29.64	21.98	31.40	29.85	25.96	30.73 (↑29.89%)
CS	Java	33.09	**34.17**	26.38	24.22	17.35	20.54	26.78	25.83	29.34	26.41
CS+D	Java	40.30	**40.57**	35.96	33.73	17.39	22.28	34.87	30.43	34.32	32.21 (↑21.94%)
CS+D+C	Java	**48.03**	45.11	37.35	34.95	30.03	30.84	41.07	31.58	40.67	38.24 (↑44.80%)

smaller models can surpass that of larger models. For example, Codellama-7B, when combined with full contextual information, achieves a CodeBLEU score of 49.53, while Codellama-34B's performance with basic conditions is only 27.54.

We attribute the performance improvement of LCMs to two main reasons: (1) *Dependency information can enhance model performance.* Specifically, import information include third-party library dependencies imported by the code file, while file-level context includes not only third-party library dependencies but also function definitions within the code file (i.e., file-level dependencies). Function dependencies encompass these two types of dependency information as well as potential cross-file dependencies within the project. These three types of dependency information can significantly enhance the performance of LCMs in complex development scenarios. (2) *Function background information can improve model performance.* Specifically, file context contains the logic and structure of the entire file's code, as well as background information on the function's purpose. This background information enables the model to understand the code task more comprehensively, thus improving performance.

Code Completion. As shown in Table 5, under basic context conditions, the average BLEU scores for Python and Java are only 12.72 and 23.04, with ES scores of 23.66 and 26.41, respectively. This indicates that merely providing code snippets (i.e., the preceding context of functions) makes it difficult for LCMs to infer the overall logic of the code. Therefore, after adding docstrings, the average BLEU scores for Python and Java increase by 33.25% and 31.38%, respectively, with ES scores improving by 13.71% and 21.94%. Furthermore, after adding file context, the average BLEU scores increase by 84.12% and 77.44%, with ES scores improving by 29.89% and 44.80%. These improvements also demonstrate that combining more contextual information can obviously enhance the performance of LCMs in code completion.

Finding 2: Rich contextual information significantly enhances the performance of LCMs in complex development scenarios by incorporating function dependency information and background information. Furthermore, our experimental results indicate that the inclusion of extensive context can enable smaller models to outperform their larger counterparts.

5.3 RQ3: Effect of Data Leakage on LCM Effectiveness

To investigate the impact of data leakage on the performance of LCMs in code generation and code completion, we select data from two timestamps based on the knowledge cut-off dates of the LCMs (as shown in Table 2): file created on or after September 15, 2023 (non-leaked data), and file created before January 1, 2023 (potentially leaked data). Based on the findings from RQ2, we employ code snippets, docstrings, and file context for code completion, while utilizing function signatures, docstrings, and file context for code generation in this section.

Data leakage. To more intuitively demonstrate the existence of data leakage, we introduce the Exact Match (EM) metric. Since each function in ComplexCodeEval contains many external dependencies and may also include variable definitions and other information, these external dependencies and variable information are unknown to LCMs during code generation. If LCMs have not seen the source code, it is almost impossible to generate code that exactly matches the reference code. However, in code completion, since the given code snippet may already include all external dependency calls and variable definitions, exact matches can occur.

As shown in Table 6, in code generation, the EM ratio for all LCMs on non-leaked data is 0, while on leaked data, the average EM ratios for LCMs in Java and Python are 0.40% and 1.06%, respectively. In code completion, the EM ratios for LCMs on non-leaked data in Java and Python are 0.13% and 0%, respectively, while the corresponding ratios on leaked data are 1.98% and 0.93%. This finding reflects the existence of data leakage issues in LCMs.

Impact of data leakage on LCMs' performance. Based on Table 6, we find that the CodeBLEU (BLEU) and ES scores of LCMs

Table 6: Performance of LCMs in code generation and code completion on data from different time periods. Before and After indicate data before January 1, 2023 and data after September 15, 2023, respectively. Bold numbers indicate the difference between before and after.

LCMs	size	Code Generation									Code Completion								
		CodeBLEU			ES			EM			BLEU			ES			EM		
		After	Before	Δ	After	Before	Δ	After	Before	Δ	After	Before	Δ	After	Before	Δ	After	Before	Δ
Python																			
DSC	1.3B	26.02	29.96	**(+3.94)**	29.95	33.27	**(+3.32)**	0.00	1.19	**(+1.19)**	16.38	22.81	**(+6.43)**	25.04	29.48	**(+4.44)**	0.00	1.19	**(+1.19)**
DSC	6.7B	32.01	37.91	**(+5.90)**	36.54	40.19	**(+3.65)**	0.00	2.38	**(+2.38)**	30.63	35.88	**(+5.25)**	38.59	42.07	**(+3.48)**	0.00	3.57	**(+3.57)**
DSC	33B	34.80	34.59	**(-0.21)**	39.64	39.88	**(+0.24)**	0.00	1.19	**(+1.19)**	28.87	32.69	**(+3.82)**	37.55	36.11	**(-1.44)**	0.00	0.00	**(0.00)**
SC2	3B	30.63	32.70	**(+2.07)**	33.95	34.57	**(+0.62)**	0.00	1.19	**(+1.19)**	15.92	22.56	**(+6.64)**	21.98	24.88	**(+2.90)**	0.00	0.00	**(0.00)**
SC2	7B	28.77	31.48	**(+2.71)**	33.08	32.98	**(-0.10)**	0.00	1.19	**(+1.19)**	23.37	29.13	**(+5.76)**	29.64	31.31	**(+1.67)**	0.00	0.00	**(0.00)**
SC2	15B	35.34	35.54	**(+0.20)**	39.39	40.74	**(+1.35)**	0.00	0.00	**(0.00)**	24.63	32.17	**(+7.54)**	33.09	35.64	**(+2.55)**	0.00	0.00	**(0.00)**
CL	7B	29.72	33.47	**(+3.75)**	34.10	37.18	**(+3.08)**	0.00	0.00	**(0.00)**	19.85	26.76	**(+6.91)**	25.96	30.40	**(+4.44)**	0.00	1.19	**(+1.19)**
CL	13B	26.49	32.20	**(+5.71)**	31.98	37.65	**(+5.67)**	0.00	1.19	**(+1.19)**	20.79	31.93	**(+11.14)**	29.85	35.43	**(+5.58)**	0.00	1.19	**(+1.19)**
CL	34B	31.52	35.36	**(+3.84)**	35.94	41.51	**(+5.57)**	0.00	1.19	**(+1.19)**	25.03	37.37	**(+12.34)**	31.40	40.10	**(+8.70)**	0.00	1.19	**(+1.19)**
Average	-	30.59	33.69	**(+3.10)**	34.95	37.55	**(+2.60)**	0.00	1.06	**(+1.06)**	22.83	30.14	**(+7.31)**	30.34	33.94	**(+3.59)**	0.00	0.93	**(+0.93)**
Java																			
DSC	1.3B	36.94	35.42	**(-1.52)**	36.04	36.71	**(+0.67)**	0.00	0.00	**(0.00)**	40.56	38.97	**(-1.59)**	37.35	38.65	**(+1.30)**	0.00	3.57	**(+3.57)**
DSC	6.7B	47.21	43.13	**(-4.08)**	43.95	43.08	**(-0.87)**	0.00	0.00	**(0.00)**	51.11	52.35	**(+1.24)**	45.11	49.68	**(+4.57)**	0.00	2.38	**(+2.38)**
DSC	33B	46.90	49.58	**(+2.68)**	45.41	50.81	**(+5.40)**	0.00	2.38	**(+2.38)**	51.96	50.67	**(-1.29)**	48.03	48.44	**(+0.41)**	0.00	0.00	**(0.00)**
SC2	3B	35.72	39.32	**(+3.60)**	35.88	41.31	**(+5.43)**	0.00	0.00	**(0.00)**	29.90	41.28	**(+11.38)**	30.84	40.27	**(+9.43)**	0.00	5.95	**(+5.95)**
SC2	7B	30.83	37.44	**(+6.61)**	31.55	37.17	**(+5.62)**	0.00	1.19	**(+1.19)**	30.09	32.10	**(+2.01)**	30.03	31.12	**(+1.09)**	1.19	0.00	**(-1.19)**
SC2	15B	46.39	45.53	**(-0.86)**	44.29	46.50	**(+2.21)**	0.00	0.00	**(0.00)**	38.89	41.81	**(+2.92)**	34.95	42.34	**(+7.39)**	0.00	0.00	**(0.00)**
CL	7B	36.45	41.48	**(+5.03)**	36.31	42.71	**(+6.40)**	0.00	0.00	**(0.00)**	45.25	47.67	**(+2.42)**	40.67	42.36	**(+1.69)**	0.00	2.38	**(+2.38)**
CL	13B	43.90	43.92	**(+0.02)**	44.36	43.77	**(-0.59)**	0.00	0.00	**(0.00)**	29.37	53.66	**(+24.29)**	31.58	46.34	**(+14.76)**	0.00	2.38	**(+2.38)**
CL	34B	46.07	45.55	**(-0.52)**	44.10	45.94	**(+1.84)**	0.00	0.00	**(0.00)**	49.51	53.27	**(+3.76)**	41.07	46.45	**(+5.38)**	0.00	1.19	**(+1.19)**
Average	-	41.16	42.37	**(+1.22)**	40.21	43.11	**(+2.90)**	0.00	0.40	**(+0.40)**	40.74	45.75	**(+5.02)**	37.74	42.85	**(+5.11)**	0.13	1.98	**(+1.85)**

on leaked data are generally higher than the corresponding scores on non-leaked data, indicating that LCMs perform better on leaked data than on non-leaked data. Specifically, as shown in Table 6, in code generation, the average CodeBLEU scores for LCMs on non-leaked data in Java and Python are 41.16 and 30.59, respectively, and the average ES scores are 40.21 and 34.95. On leaked data, the average CodeBLEU scores are 42.37 and 33.69, and the average ES scores are 43.11 and 37.55. Compared to non-leaked data, the average CodeBLEU scores on leaked data increased by 1.22 and 3.10, and the average ES scores increased by 2.90 and 2.60, respectively. In code completion, the average BLEU scores on leaked data and non-leaked data increased by 5.02 and 7.31, respectively, and the average ES scores increased by 5.11 and 3.59, respectively. These results suggest that data leakage may affect the performance of LCMs to some extent, leading to an overestimation of LCMs' performance. Additionally, since more information is provided to the model in code completion, the risk of overestimation is higher. The performance increase on leaked data compared to non-leaked data is also inconsistent across different models. For example, in Java code completion, the BLEU increase for StarCoder2-3B is 11.38, while for StarCoder2-7B it is only 2.01, indicating that using leaked data may not accurately assess the model's performance.

Finding 3: LCMs exhibit superior performance on leaked data compared to non-leaked data, with the degree of performance improvement due to data leakage varying across different models. Consequently, evaluating LCMs' performance on code using leaked data may result in inaccurate and biased outcomes, failing to accurately reflect the models' true performance in real-world applications.

6 Discussion

6.1 Performance with Different Docstrings

We use an LCM (DeepSeek-Coder-33B in this paper) to automatically generate docstrings for functions. We evaluate the LCMs using both the original docstrings and the automatically generated docstrings in this section. To avoid the influence of context on the results, we choose to evaluate Python code generation in the base context (i.e., including only the docstring and function signature). Table 7 shows the CodeBLEU and Edit Similarity (ES) scores for different LCMs when using original and automatically generated docstrings.

As seen in Table 7, the CodeBLEU and ES scores of LCMs are generally higher when using automatically generated docstrings compared to the original docstrings. Specifically, by utilizing docstrings generated by LCM, the average CodeBLEU score of LCMs reaches 27.88, and the average ES score reaches 33.50, representing respective increases of 3.33 and 4.00. This phenomenon can be attributed to two main reasons. First, automatically generated docstrings are typically more detailed and accurate than the original docstrings, providing the LCMs with comprehensive information about the functions that help the model better understand and generate code. Second, the quality of original docstrings varies widely, with many being relatively short and unclear, limiting the model's performance in code generation tasks. In contrast, automatically generated docstrings, which are carefully designed and iteratively optimized, can more accurately describe the target functions, considerably enhancing the model's performance.

Table 7: Effectiveness with two prompt versions of ComplexCodeEval. OD and GD indicate old docstring and generated docstring , respectively. Bold numbers indicate the difference between GD and OD.

LCMs	Size	CodeBLEU			ES		
		OD	GD	Δ	OD	GD	Δ
DSC	33B	27.61	29.93	**(+2.32)**	31.51	34.75	**(+3.24)**
DSC	6.7B	27.99	30.77	**(+2.78)**	33.40	35.92	**(+2.52)**
DSC	1.3B	20.99	23.60	**(+2.61)**	25.92	28.26	**(+2.34)**
SC2	15B	28.62	29.77	**(+1.15)**	33.72	36.32	**(+2.60)**
SC2	7B	21.85	29.41	**(+7.56)**	26.56	35.26	**(+8.70)**
SC2	3B	21.92	26.38	**(+4.46)**	27.18	31.22	**(+4.04)**
CL	34B	27.89	30.69	**(+2.80)**	32.56	36.24	**(+3.68)**
CL	13B	19.03	22.43	**(+3.40)**	24.56	29.53	**(+4.97)**
CL	7B	25.07	27.98	**(+2.91)**	30.12	34.02	**(+3.90)**
Average	-	24.55	27.88	**(+3.33)**	29.50	33.50	**(+4.00)**

Table 8: Performance of Codellama-34B on different benchmark.

Metric	Benchmark	Python	Java
Code Generation			
CodeBLEU	CoderEval	26.38	35.88
	ComplexCodeEval	27.54	34.08
Edit Similarity	CoderEval	33.27	42.71
	ComplexCodeEval	32.93	36.93
Code Completion			
BLEU	CrossCodeEval	39.98	40.51
	ComplexCodeEval	13.79	22.36
Edit Similarity	CrossCodeEval	62.76	62.84
	ComplexCodeEval	25.48	26.78
Test Case Generation			
CodeBLEU	Method2Test	-	35.79
	ComplexCodeEval	-	29.90
Edit Similarity	Method2Test	-	30.15
	ComplexCodeEval	-	29.39
API Recommendation			
F1	APIbench	45.38	29.54
	ComplexCodeEval	52.24	38.92
Recall	APIbench	45.84	29.72
	ComplexCodeEval	52.14	38.73

Overall, these results indicate that using high-quality docstrings is crucial for improving the performance of LCMs in code generation tasks. Future research can further explore how to optimize the process of generating docstrings to further enhance the performance of LCMs in code-related tasks.

6.2 Performance With Different Benchmark

In Section 5.1, the experimental results demonstrate that Codellama-34B achieves the best performance across four tasks. Consequently, this section focuses on Codellama-34B as the subject of our experiments. As illustrated in Table 8, we compare performance in code generation using CoderEval [35]. Due to the high dependency on samples in CoderEval, Codellama-34B performs suboptimally on both CoderEval and ComplexCodeEval. In code completion, Codellama-34B performs better on CrossCodeEval [9], which focuses on simpler line-level completions, than on ComplexCodeEval. For test case generation, Codellama-34B exhibits slightly superior performance on Method2Test [31] compared to ComplexCodeEval, whereas in API recommendation, Codellama-34B performs better on ComplexCodeEval than on APIbench [27].

These results highlight the differences between ComplexCodeEval and various benchmarks associated with different tasks. Our benchmark not only encompasses multiple tasks but also illustrates performance disparities with existing benchmarks, thereby validating the necessity of introducing ComplexCodeEval.

6.3 Threats To Validity

Threats in Benchmark Construction. One potential threat is the way how we record the timestamps of the selected repositories. In order to mitigate the risk of not obtaining the corresponding time information from git commit records, we adopt a fallback mechanism: when the file creation time is unavailable, we use the project creation time as the file creation time; when the update time is unavailable, we use the creation time as the update time. This mechanism ensures the reliability and robustness of the samples.

Threats in Empirical Study. Due to computing resource constraints, we do not conduct experiments on more open-source models (e.g., CodeGeex [36], WizardCoder [25]) and closed-source models (e.g., Gemini [8]). Thus we select nine open-source LCMs ranging from 1.3B to 34B parameters and one closed-source LCM for experimentation, thereby covering a wide spectrum of model complexity and capability.

7 Conclusion

In this paper, we present ComplexCodeEval, a novel benchmark designed to evaluate the performance of LCMs in complex development scenarios. Unlike existing benchmarks, ComplexCodeEval is more adaptable and leverages a rigorous automated pipeline for data collection. Our experimental results highlight the limitations of LCMs in handling complex development tasks, while demonstrating that providing rich contextual information can substantially improve their performance. Furthermore, by comparing LCMs performance across data from different time periods, we emphasize that the use of leaked data in evaluations can lead to inaccurate results and introduce bias.

Acknowledgment

This research is supported by the National Natural Science Foundation of China under project (No. 62472126), Natural Science Foundation of Guangdong Province (Project No. 2023A1515011959), Shenzhen-Hong Kong Jointly Funded Project (Category A, No. SGDX20230116091246007), Shenzhen Basic Research (General Project No. JCYJ20220531095214031), Shenzhen International Science and Technology Cooperation Project (No. GJHZ20220913143008015).

References

[1] 2024. https://github.com/ComplexCodeEval/ComplexCodeEval.
[2] 2024. GitHub Copilot. https://github.com/features/copilot.
[3] Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Munoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu,

Manan Dey, et al. 2023. SantaCoder: don't reach for the stars! *arXiv preprint arXiv:2301.03988* (2023).

[4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

[5] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. 2023. MultiPL-E: a scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering* (2023).

[6] Shubham Chandel, Colin B Clement, Guillermo Serrato, and Neel Sundaresan. 2022. Training and evaluating a jupyter notebook data science assistant. *arXiv preprint arXiv:2201.12901* (2022).

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[8] Google DeepMind. 2023. Gemini: A multi-modal large language model. https://www.deepmind.com/research/gemini. Accessed: [Date].

[9] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2023. CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. https://openreview.net/forum?id=wgDcbBMSfh

[10] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. *arXiv preprint arXiv:2308.01861* (2023).

[11] Association for Computational Linguistics. Meeting, Araving Joshi, and Martha Palmer. 1996. Proceedings of the 34th Annual Meeting on Association for Computational Linguistics: Santa Cruz (California), June 24-27, 1996. Association for Computational Linguistics.

[12] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2024. Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[13] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming–The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[14] Patrick Haluptzok, Matthew Bowers, and Adam Tauman Kalai. 2022. Language models can teach themselves to program better. *arXiv preprint arXiv:2207.14502* (2022).

[15] Yiyang Hao, Ge Li, Yongqiang Liu, Xiaowei Miao, He Zong, Siyuan Jiang, Yang Liu, and He Wei. 2022. Aixbench: A code generation benchmark dataset. *arXiv preprint arXiv:2206.13179* (2022).

[16] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. 2021. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938* (2021).

[17] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2018. Mapping language to code in programmatic context. *arXiv preprint arXiv:1808.09588* (2018).

[18] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. *arXiv preprint arXiv:2403.07974* (2024).

[19] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning*. PMLR, 18319–18345.

[20] Jia Li, Ge Li, Xuanming Zhang, Yihong Dong, and Zhi Jin. 2024. EvoCodeBench: An Evolving Code Generation Benchmark Aligned with Real-World Code Repositories. *arXiv preprint arXiv:2404.00599* (2024).

[21] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).

[22] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.

[23] Libraries.io. 2024. Libraries.io: The Open Source Discovery Service. https://libraries.io.

[24] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. *arXiv preprint arXiv:2402.19173* (2024).

[25] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568* (2023).

[26] Phuong T Nguyen, Juri Di Rocco, Davide Di Ruscio, Lina Ochoa, Thomas Degueule, and Massimiliano Di Penta. 2019. Focus: A recommender system for mining api function calls and usage patterns. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 1050–1060.

[27] Yun Peng, Shuqing Li, Wenwei Gu, Yichen Li, Wenxuan Wang, Cuiyun Gao, and Michael Lyu. 2021. Revisiting, Benchmarking and Exploring API Recommendation: How Far Are We? arXiv:2112.12653 [cs.SE]

[28] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. arXiv:2009.10297 [cs.SE]

[29] Tal Ridnik, Dedy Kredo, and Itamar Friedman. 2024. Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering. *arXiv preprint arXiv:2401.08500* (2024).

[30] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[31] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit Test Case Generation with Transformers and Focal Context. arXiv:2009.05617 [cs.SE]

[32] Thomas Wang, Adam Roberts, Daniel Hesslow, Teven Le Scao, Hyung Won Chung, Iz Beltagy, Julien Launay, and Colin Raffel. 2022. What language model architecture and pretraining objective works best for zero-shot generalization?. In *International Conference on Machine Learning*. PMLR, 22964–22984.

[33] Zhiruo Wang, Shuyan Zhou, Daniel Fried, and Graham Neubig. 2022. Execution-based evaluation for open-domain code generation. *arXiv preprint arXiv:2212.10481* (2022).

[34] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2023. Magicoder: Source code is all you need. *arXiv preprint arXiv:2312.02120* (2023).

[35] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.

[36] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, et al. 2023. Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x. *arXiv preprint arXiv:2303.17568* (2023).

[37] Maosheng Zhong, Zhixiang Wang, Gen Liu, Youde Chen, Huizhu Liu, and Ruping Wu. 2023. Codegen-test: An automatic code generation model integrating program test information. In *2023 2nd International Conference on Cloud Computing, Big Data Application and Software Engineering (CBASE)*. IEEE, 341–344.